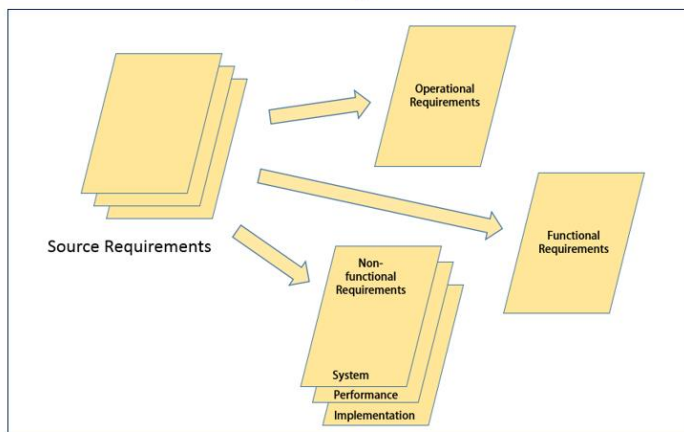


Tutorial Exercise 5

Textual & Viewpoint Analysis: Assessment project

1. Apply Systemic Textual Analysis for your project's requirements. Can you identify anything that is missing?
2. You could also apply the Viewpoint Analysis for your project requirements. Remember that the idea is to capture the requirements as fully as possible, to categorize them, group them and produce a hierarchy that shows the relationships between the functional requirements. Your diagram should also show how the non-functional requirements relate to the functional ones.
3. Once you have analysed the requirements using the methods above, apply the MoSCoW method.
4. Check whether your revised set of requirements matches the characteristics of good ones. Produce a table that acts as a check list for this.
5. If you have identified any new non-functional system or performance requirements, think about how you would go about measuring them. This is key to good testing.
6. Have a reflective review of your progress, and which requirements methods might be most effective for your particular product.
7. Review your tentative deliverable 1 document "Requirements Analysis" for improvement.

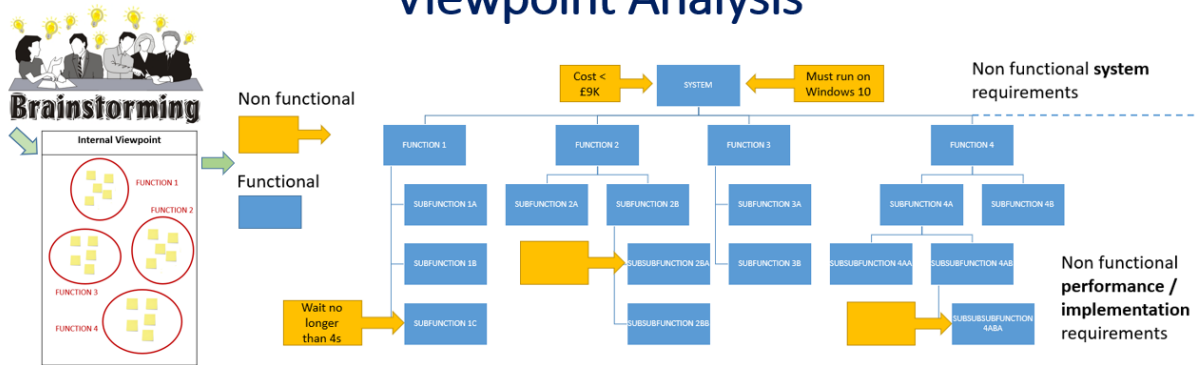
Systemic Textual Analysis



- Allows us to identify **missing** and **ambiguous** requirements
- **No relationships** between the different functional requirements

Systemic Textual Analysis			
Project:		Date:	
Author:		Issue:	
Requirements		Comments	
Context:			
Operational Requirement:			
Non-functional System Requirements:			
Non-functional Implementation Requirement	Functional Requirement	Non-functional Performance Requirement	

Viewpoint Analysis



1. Brainstorm
2. Separate requirements into **functional / non-functional**
3. Separate requirements into **internal / external**
4. Group **functional** requirements and name
5. Draw groups as a **hierarchical chart**
6. Add in the **non-functional** requirements

- Allows us to identify **missing** and **ambiguous** requirements
- **Relationships** between functions and sub-functions but not between top level functions

Characteristics of Good Requirements

Characteristic	Explanation
Unitary (Cohesive)	The requirement addresses one and only one thing.
Complete	The requirement is fully stated in one place with no missing information.
Consistent	The requirement does not contradict any other requirement and is fully consistent with all authoritative external documentation.
Non-Conjugated (Atomic)	The requirement is <i>atomic</i> , i.e., it does not contain conjunctions. E.g., "The postal code field must validate American <i>and</i> Canadian postal codes" should be written as two separate requirements: (1) "The postal code field must validate American postal codes" and (2) "The postal code field must validate Canadian postal codes".
Traceable	The requirement is authoritatively documented.
Current	The requirement has not been made obsolete by the passage of time.
Unambiguous	The requirement is concisely stated without recourse to technical jargon, acronyms (unless defined elsewhere in the Requirements document), or other esoteric verbiage. It expresses objective facts, not subjective opinions. It is subject to one and only one interpretation. Vague subjects, adjectives, prepositions, verbs and subjective phrases are avoided. Negative statements and compound statements are avoided.
Specify Importance	Many requirements represent a stakeholder-defined characteristic the absence of which will result in a major or even fatal deficiency. Others represent features that may be implemented if time and budget permits. The requirement must specify a level of importance.
Verifiable	The implementation of the requirement can be determined through basic possible methods such as inspection, demonstration, test or analysis

Based on: Davis, Alan M. (1993). *Software Requirements: Objects, Functions, and States*, Second Edition. Prentice Hall.

Requirements Prioritisation: MoSCoW Method

A common system for **prioritising requirements**

- **M – Must.** These are *required* features that **must be included**. They are usually necessary for the application to function correctly. Without them the project would be considered a failure
- **S – Should.** These are *important* features that **should be included** if possible.
- **C – Could.** These are *desirable* features that **could be omitted** if the schedule or budget is too tight for them. They are not as important as 'should' features
- **W - Won't.** These are *optional* features that the customers have agreed will not be included in the current release. They may be **included in future releases** if time permits.

